

CS1910

Tuples

2D Lists

List Comprehensions

TUPLES

- An immutable sequence of data
 - Like lists but immutable
 - Ordered, allows any type of data (like lists)
 - Specified with ()
 - Access elements at index positions with []
 - Useful in places where you want something that is immutable
 - Stay tuned for Dictionaries where there is an obvious use for immutable sequences (tuples) as opposed to lists

```
01 # Empty tuple
02 # Useful when you need a placeholder for an empty variable
03 # That will not change
04 empty_tuple = ()
05 # Tuple with mixed data types
06 # Can hold multiple types like lists
07 my_tuple = ("apple", 1, 3.14, True)
08 # Tuple with a single element (note the comma)
09 # Comma is needed or otherwise it is just a string
10 single_element_tuple = ("hello",)
11 # Accessing by index
12 print(my_tuple[0]) # Output: apple
13 print(my_tuple[2]) # Output: 3.14
14 # Slicing
15 print(my_tuple[1:3]) # Output: (1, 3.14)
```

MULTI-DIMENSIONAL LISTS

- Up until recently we have had inner lists but no other real restrictions
 - Recall that accessing an inner list uses the syntax:

```
1 lst = [ [1, 2, 3], [5, 4, 6], [7, 8, 9] ]  
2  
3 # accessing individual elements  
4 lst[1][2] # 2nd index position of the 1st element in lst
```
- This allows us to work in two dimensions if we wish – allowing us to mimic matrices or other 2D structures like images, graphs or really anything.
- We can even push into further dimensions – one could keep the points of triangles for a character model in a game or movie by nesting 3 lists.

ITERATING MULTIDIMENSIONAL LISTS

```
01 #If we do not need indices - for ... in ... syntax
02 print("for each")
03 #using for each loop
04 for sub_lst in lst:
05     for value in sub_lst:
06         print(value, end = " ") # " " replacing \n
07     print() # \n add \n to the row
```

Conceptually: Assuming [[5 ,5, 7], [5, 5, 4], [5, 5, 1]] is our list ...

1. Get the next item in the list, which is a list
 - a) Get the next item in the retrieved inner list
 - b) Repeat (a) until there are no other items
2. Repeat 1 until there are no other items in the outer list

CREATING AND USING MULTIDIMENSIONAL LISTS

- Manipulating multi-dimensional lists through builtins requires identifying how typical functions apply to different data
- For example – with sorting on 2D lists, it works similar to,, but notably different from, other types:
 - We still apply lexicographic order
 - We need to apply it to each sub list
 - We start with:
 - Compare index 1 of each sublist
 - Compare index 2 of each sublist
 - ...
 - Compare index n of each sublist
- For different complex types, you will apply similar rules – so if it was lists of strings, the lexicographic order would apply with the characters

```
01 lst = [ [3, 2, 1], [6, 5, 4], [9, 8, 7] ]
02 # Sort elements of lst - retrieve [6,5,4] and sort it
03 lst[1].sort()
04 print(lst) # [[3, 2, 1], [4, 5, 6], [9, 8, 7]]
05 # Sort the overall list
06 lst.sort()
07 # List has been compared lexicographically - 3 < 4 < 9
08 print(lst) # [[3, 2, 1], [4, 5, 6], [9, 8, 7]]
09 # New list with different lexicographic order
10 lst = [[6, 5, 4], [3, 2, 1], [9, 8, 7]]
11 lst.sort()
12 print(lst) # [[3, 2, 1], [6, 5, 4], [9, 8, 7]]
13 # Extreme example where only the last item matters
14 lst = [[5, 5, 4], [5, 5, 1], [5, 5, 7]]
15 lst.sort()
16 print(lst) # [[5, 5, 1], [5, 5, 4], [5, 5, 7]]
17 # We can put it back with an optional reverse parameter
18 lst.sort(reverse=True)
19 print(lst) # [[5, 5, 7], [5, 5, 4], [5, 5, 1]]
```

ITERATING MULTIDIMENSIONAL LISTS

```
01  #If we do not need indices - for ... in ... syntax
02  print("for each")
03  #using for each loop
04  for sub_lst in lst:
05      for value in sub_lst:
06          print(value, end = " ") # " " replacing \n
07          print() # add \n to the row
08  # For Each
09  # 5 5 7
10  # 5 5 4
11  # 5 5 1
12
13  # If we need indices in the code we must use ranges
14  print("Using indices")
15  #using indices
16  for i in range(len(lst)):
17      for j in range(len(lst[i])):
18          print(lst[i][j], end = " ") # " " replacing \n
19          print() # add \n to the row
20  # Using indices
21  # 5 5 7
22  # 5 5 4
23  # 5 5 1
```

FIND A PARTNER

- Grayscale images are 2D lists (height x width) where each pixel in the image is a value between 0 and 255 giving the intensity of the pixel (0 is black, 255 is white and values in between are scales of gray).

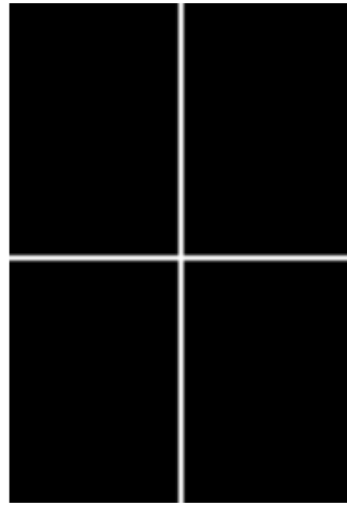
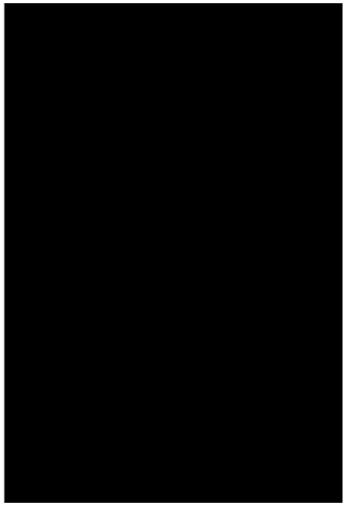
3 x 3 image

```
image = [ [pixel, pixel, pixel],  
          [pixel, pixel, pixel],  
          [pixel, pixel, pixel]]
```

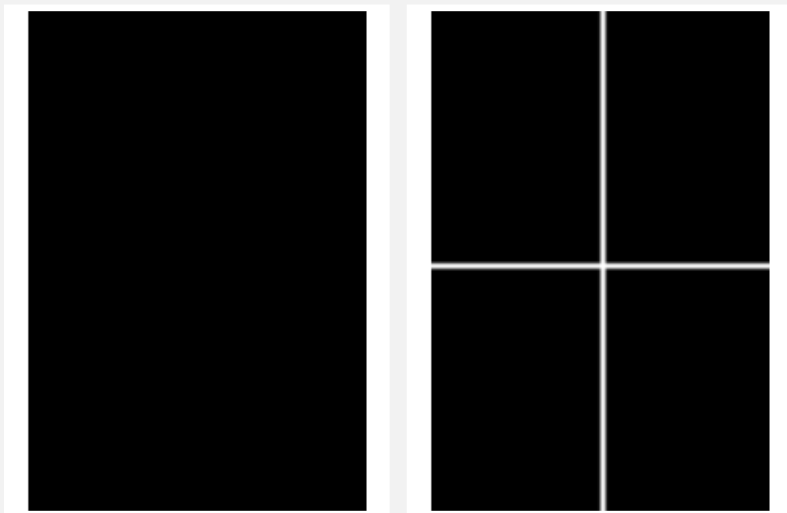


Example grayscale image of a sailor eating spinach from a can [source: youtube]

FIND A PARTNER



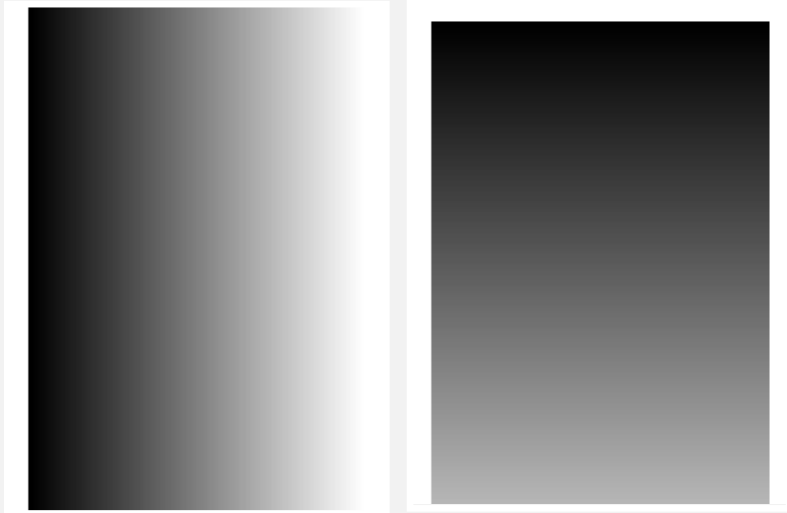
- 4 Quadrant Image
 - What details matter and what can we ignore?
 - Focus on:
 - numeric pixel values
 - relative position in the grid
 - Abstraction question for students:
 - If I didn't know this was an image, what patterns could I still find in the numbers?



FIND A PARTNER

- Image dimensions and indexing
 - Your image has:
 - Height = 128 rows → indexed 0 to 127
 - Width = 64 columns → indexed 0 to 63
 - Python uses 0-based indexing, which is key here.
 - For odd dimensions, there is a single centre index:
 - Height = 5 → indices 0 1 2 3 4 → centre is index 2
 - For even dimensions, there is no single centre index:
 - Height = 128 → indices 0 ... 63 | 64 ... 127
 - Width = 64 → indices 0 ... 31 | 32 ... 63
 - The vertical and horizontal midpoints lie between:
 - Rows 63 and 64 and Columns 31 and 32
 - So any choice of 63 or 64 (or 31 or 32) is one cell off the centre.
 - So we need to choose – will be bias towards even or odd

FIND A PARTNER



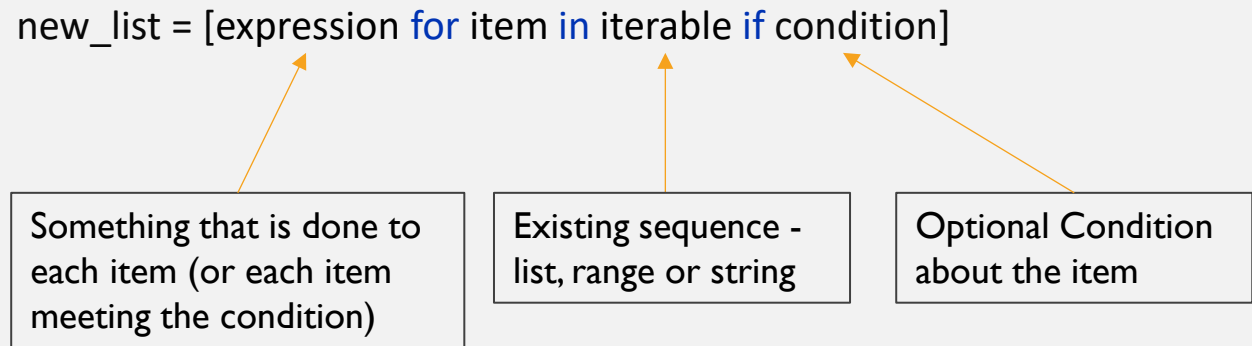
- Gradient Image
 - Decomposition – similar to previous question
 - Identify number, then populate lists
 - Patterns: Two versions of the same problem
 - Create a 128x64 gradient image
 - The gradient fills from left to right and black to white
 - The gradient fills from top to bottom
 - Hypothetical: The gradient fills from top left corner to bottom right corner
 - If I didn't know this was an image, what patterns could I still find in the numbers?
 - Generalization question:
 - Is there a way that you can create both of these images with 1 function?

LIST COMPREHENSIONS

- Concise way to create new lists based on existing lists (or other sequences)
- Alternative to using for loops to create new lists from other lists (or sequences)
- Syntax:

`new_list = [expression for item in iterable if condition]`

Something that is done to
each item (or each item
meeting the condition)



```
graph BT; A[Something that is done to each item (or each item meeting the condition)] --> B[expression]; B --> C[for item]; C --> D[in iterable]; D --> E[if condition]; E --> F[Optional Condition about the item];
```

Existing sequence -
list, range or string

Optional Condition
about the item

EXAMPLE LIST COMPREHENSION

Example: Integer list

```
1  # Create a new list of squares based on existing list
2  numbers = [1, 2, 3, 4, 5]
3  #use a list comprehension to generate squares of the original values
4  squares = [x**2 for x in numbers]
5  # squares will be [1, 4, 9, 16, 25]
```

Example: String manipulation

```
1  # Create a new list with words over length 5
2  words = ["apple", "banana", "cherry", "date"]
3  #list of long words from the existing list
4  long_words = [word for word in words if len(word) > 5]
5  # long_words will be ["banana", "cherry"]
```

EXAMPLES

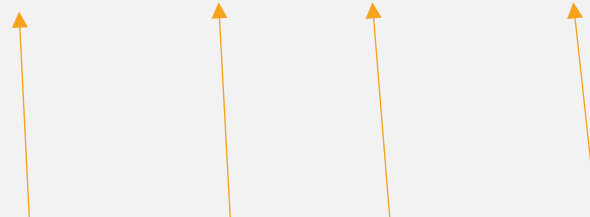
#Create a new list with words over length 5

words = ["apple", "banana", "cherry", "date"]

#list of long words from the existing list

long_words = [word for word in words if len(word) > 5]

new_list = [expression for item in iterable if condition]



EXAMPLES

#Create a new list with words over length 5

words = ["apple", "banana", "cherry", "date"]

#list of long words from the existing list

long_words = [word for word in words if len(word) > 5]

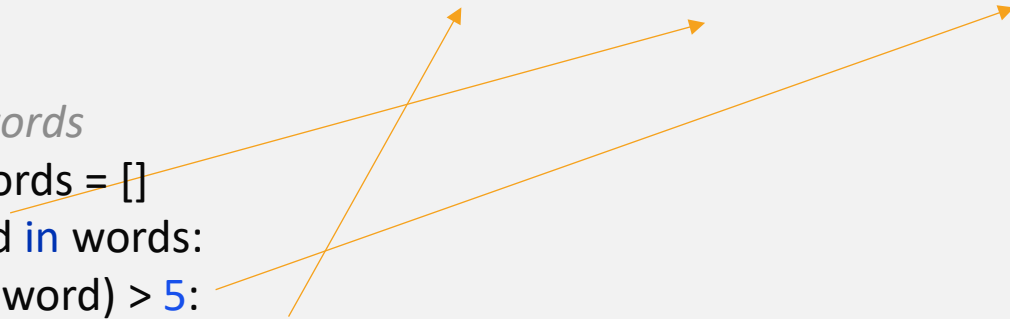
1 *#long words*

2 long_words = []

3 for word in words:

4 if len(word) > 5:

5 long_words.append(word)



EXAMPLE LIST COMPREHENSION

Example: Integer list

```
1 # Create a new list of squares based on existing list
2 numbers = [1, 2, 3, 4, 5]
3 #use a list comprehension to generate squares of the
  original values
4 squares = [x**2 for x in numbers]
```

Example: String manipulation

```
1 # Create a new list with words over length 5
2 words = ["apple", "banana", "cherry", "date"]
3 #list of long words from the existing list
4 long_words = [word for word in words if len(word) > 5]
```

Without comprehensions:

```
1 # square of numbers
2 squares = []
3 for num in numbers:
4     squares.append(num**2)
```

```
1 #long words
2 long_words = []
3 for word in words:
4     if len(word) > 5:
5         long_words.append(word)
```

SELF CHECK

- Which of the below is a list comprehension from line 08 - 11 to select only the words containing “pk”?

```
01  # Select words containing "pk" letter combination using a list comprehension
02  # Reword the problem:
03  # For each word in words check the following:
04  # Is "pk" in w True or False - add the answer to the list
05  # Which of the following is correct?
06  words = ["napkin", "pumpkin", "dropkick", "country", "bumpkin", "bubkes", "shopkeeper", "topknot",
07           "upkeep", "slipknot", "shipknot"]
08  pk_words = [for w in words if "pk" in words]
09  pk_words = [w for w in words if "pk" in w]
10  pk_words = ["pk" in w for w in words]
11  pk_words = [w.index("pk") for w in words if "pk" in words]
```


FIND A PARTNER

Complete each of the comprehensions.

Before writing the comprehension, try to explain what the steps are it must do in terms of:

What set of data is the expression operating over?

What conditions on the data constrain when the expression is applied?

Find a Partner - Comprehensions

```
numbers = range(100)
```

Complete 7 tasks using list comprehensions

#1. all the numbers in 0-99 that are divisible by 7

#2. the upper case version of a list of words

```
words = ["lower", "case", "version", "words"]
```

#3. convert a list of strings that represent integers into actual integers (type int)

```
number_strings = ["1", "123", "456", "789"]
```

#4. the square root of all the numbers in 0-999 that are divisible by 77

#5. convert a list of strings into a list of ints giving the lengths of those strings

```
strings = ["a", "aa", "aaa", "aaaa"]
```

#6 Convert a list of numbers between 0 and 53 into their percentage out of 53

```
grades = [49, 47, 23, 32, 44, 52]
```

#7 Convert a list of floating point numbers to a measure of how close they are to math.pi

```
floats = [3.14, 22/7, 355/113, 3.0, 25/8]
```



TRY AT HOME

- Print out all the elements in a multidimensional list
- Remove specific elements from the list
- Write a method to create and return images
- Search for ghosts in a pacman game board